

NAVIGATORS — COMPLETE ENGINEERING FIX, INTEGRATION & VALIDATION TASK

Project

Project Name: **Navigators**

SIH Problem Statement: **SIH26168**

Organization: **Indian Space Research Organisation (ISRO)**

Objective:

Build an AI/ML-powered Intelligent Dead Reckoning (IDR) system that maintains continuous vehicle navigation during GNSS outages using smartphone IMU sensors, without relying on OBD-II or a vehicle speedometer.

The system must operate on the edge device and ultimately support 100% offline execution.

1. YOUR ROLE

Act as a **senior navigation systems engineer + ML engineer + embedded/edge engineer + software architect**.

You are working on an existing repository.

DO NOT rebuild the project blindly.

First inspect the complete repository, understand what already exists, identify what is actually implemented, identify broken/incomplete components, and then incrementally improve the existing implementation.

Your priority is:

1. Correctness
2. Navigation accuracy
3. Physical consistency
4. Robustness
5. Edge performance
6. Offline operation
7. Testability
8. Maintainability
9. UI/demo quality

Do not sacrifice navigation correctness for visual polish.

2. IMPORTANT RULE

Do NOT assume that something is implemented merely because documentation says it is implemented.

For every claimed capability:

- inspect the source code,
- inspect tests,
- run the tests,
- execute the component where possible,
- verify the output,
- measure performance,
- identify discrepancies between documentation and implementation.

Create a clear distinction between:

- Implemented and verified
- Implemented but insufficiently tested
- Partially implemented
- Placeholder/mock
- Planned but not implemented
- Broken

Do not claim success without evidence.

3. FIRST: COMPLETE REPOSITORY AUDIT

Before changing code, inspect:

```
src/  
scripts/  
simulator/  
tests/  
models/  
datasets/  
configuration files  
package files  
requirements files  
README  
documentation
```

Identify:

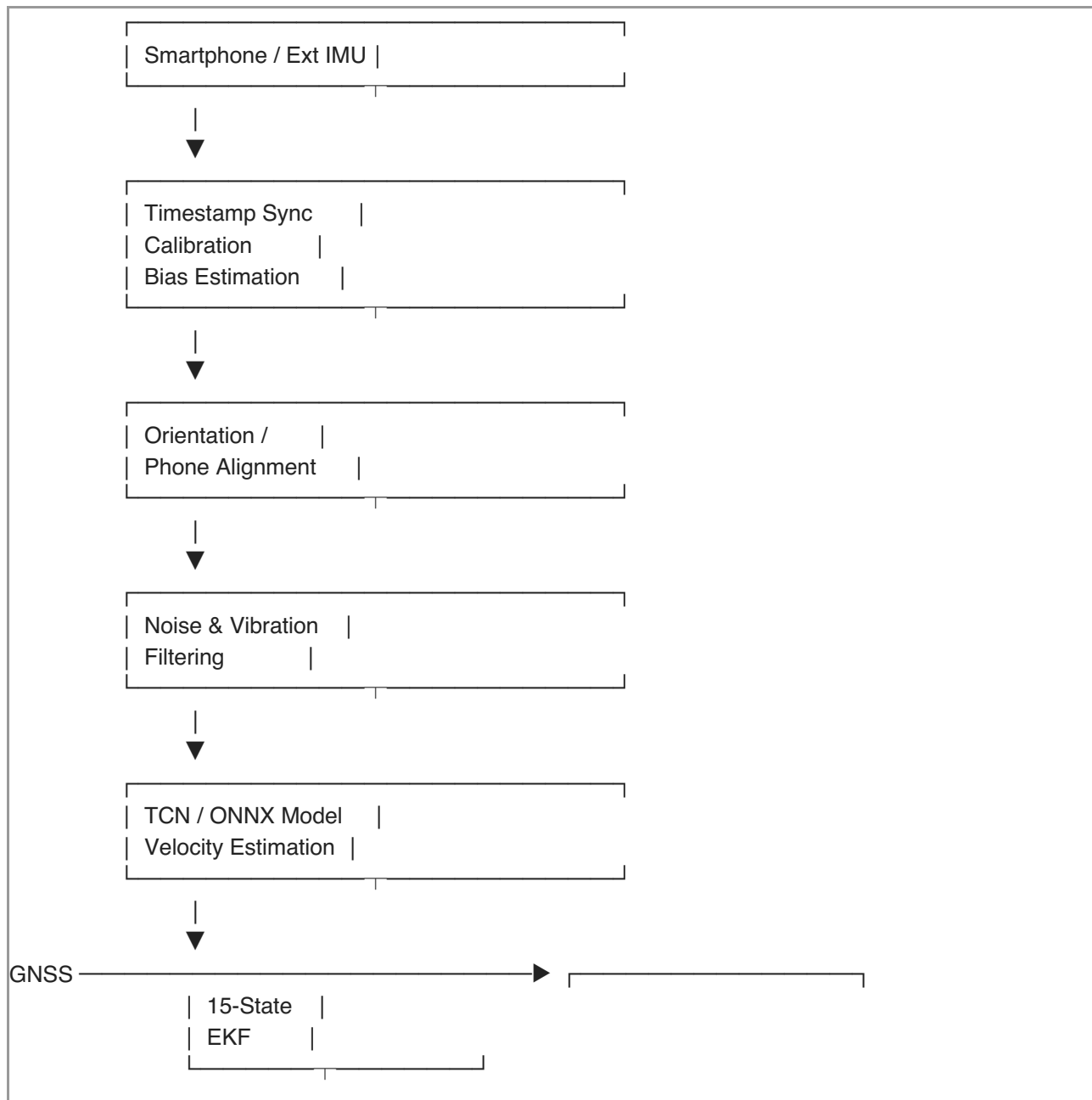
- entry points,
- data pipeline,
- model pipeline,
- preprocessing,
- sensor handling,
- coordinate transformations,
- EKF,
- dead reckoning,
- NHC,
- ZUPT,
- map matching,
- GNSS outage detection,
- GNSS recovery,

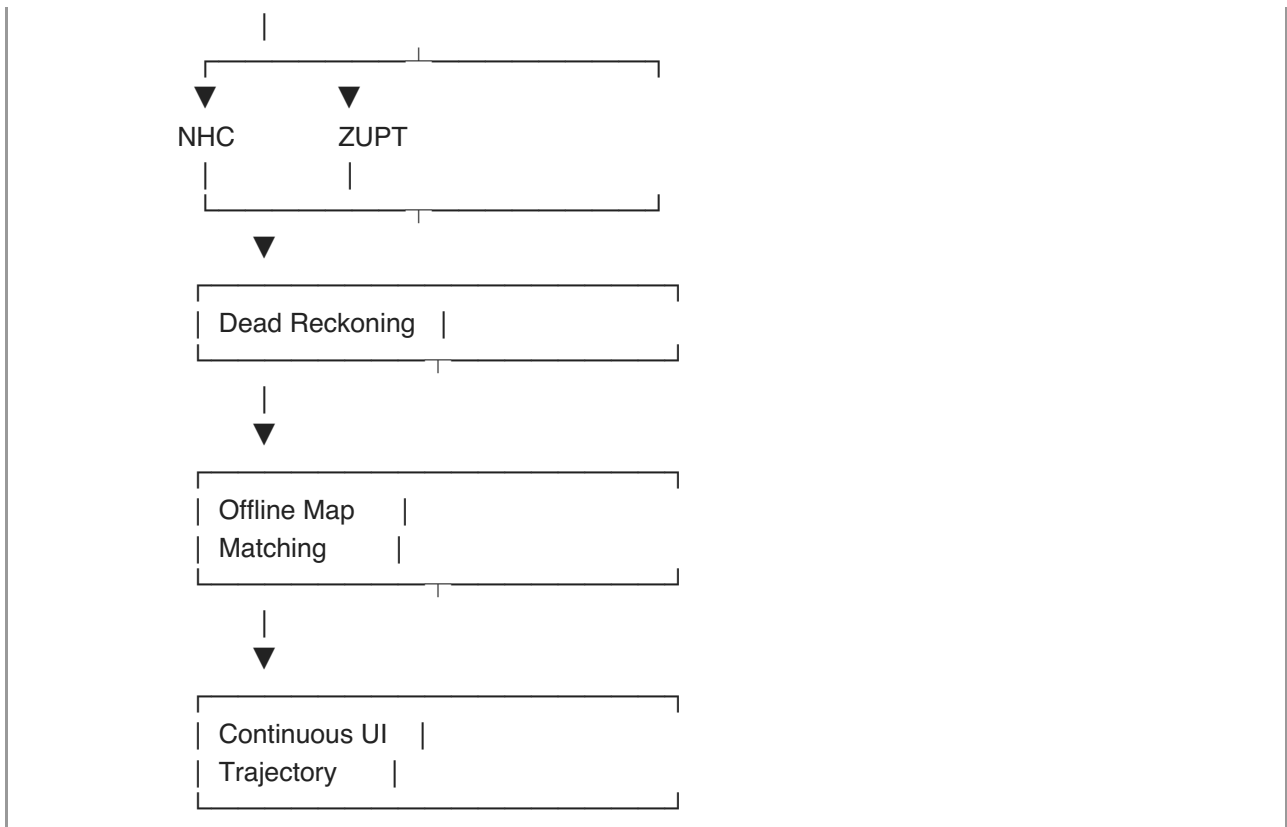
- ONNX export,
- ONNX inference,
- WebAssembly,
- PWA,
- Service Worker,
- UI,
- simulator,
- API/backend,
- tests.

Create an internal dependency map before modifying anything.

4. REQUIRED FINAL ARCHITECTURE

The final architecture should follow this conceptual pipeline:





The final runtime navigation pipeline must NOT depend on a Python/FastAPI backend.

Python may remain for:

- training,
- offline analysis,
- evaluation,
- development tools,
- dataset processing.

Live navigation must be capable of running entirely on the edge device.

5. FIX THE DATA PIPELINE

Inspect the IO-VNBD loader and dataset implementation.

Verify:

- timestamp ordering,
- duplicate timestamps,
- missing samples,
- sampling rate,
- sensor synchronization,
- GNSS synchronization,
- accelerometer units,
- gyroscope units,
- coordinate conventions,
- latitude/longitude handling,

- velocity labels,
- heading labels,
- train/validation/test leakage.

Implement robust:

timestamp synchronization
missing-data handling
outlier detection
sensor calibration
normalization
window generation
label generation

Do not silently discard problematic data.

Log how much data is removed and why.

6. PREVENT DATA LEAKAGE

Do NOT randomly split individual overlapping IMU windows if windows from the same trajectory can appear in both training and test sets.

Prefer trajectory/sequence-level separation.

The evaluation set must contain trajectories that the model did not see during training.

Where possible test:

- unseen route,
- unseen vehicle,
- different mounting position,
- different motion profile.

Document the exact split.

7. FIX PHONE-TO-VEHICLE ALIGNMENT

This is a critical requirement.

The system must handle arbitrary phone orientation.

Implement an explicit transformation:

Phone Coordinate Frame
↓
Phone-to-Vehicle Rotation
↓
Vehicle Coordinate Frame
↓
Navigation Frame

Use available:

- accelerometer,
- gyroscope,
- gravity direction,
- temporal motion information,
- vehicle-motion constraints.

Estimate:

roll
pitch
yaw

and maintain a rotation representation such as:

rotation matrix
or
quaternion

Avoid repeated Euler-angle calculations where they introduce instability.

Implement:

$R_{\text{vehicle_phone}}$

and verify:

$a_{\text{vehicle}} = R_{\text{vehicle_phone}} \times a_{\text{phone}}$

with unit tests.

Test multiple simulated phone orientations.

8. FIX IMU PREPROCESSING

Build a robust preprocessing pipeline:

Raw IMU
↓
Timestamp validation
↓
Outlier detection
↓
Bias estimation
↓
Gravity estimation/removal
↓
Orientation estimation
↓
Phone → vehicle transformation
↓
Vibration/noise filtering
↓
Feature normalization
↓
TCN

Do not blindly apply an arbitrary low-pass filter.

Preserve motion information required by the model.

Evaluate preprocessing using actual data.

Compare:

raw IMU
vs
filtered IMU

and measure its effect on velocity and trajectory error.

9. IMPROVE THE TCN

Inspect the existing TCN implementation.

Do not replace it automatically.

First establish a baseline.

Evaluate:

TCN
LSTM

only if necessary.

Primary model output should preferably represent vehicle motion such as:

forward velocity
lateral velocity

rather than directly predicting latitude/longitude.

Keep the model lightweight enough for edge inference.

Evaluate:

- MAE,
- RMSE,
- velocity error,
- heading error where applicable,
- trajectory error,
- inference latency,
- model size.

Do not optimize only for training loss.

Optimize for **GNSS-denied navigation performance**.

10. IMPROVE TRAINING

Inspect the training pipeline.

Add where appropriate:

- normalization,
- augmentation,
- noisy IMU augmentation,
- random orientation augmentation,
- vibration augmentation,
- speed variation,
- acceleration/deceleration variation,
- different window positions.

The goal is to make the model robust to:

engine vibration
road vibration
potholes
bumps
shocks
phone movement
sensor noise
sensor bias

Do not fabricate unrealistic augmentation distributions.

Keep training and test distributions clearly separated.

11. IMPLEMENT / VERIFY THE 15-STATE EKF

Inspect the existing EKF.

Verify its state definition.

Explicitly document the state vector.

Verify:

```
prediction
Jacobian
process covariance
measurement covariance
GNSS update
velocity update
innovation
innovation covariance
Kalman gain
state correction
covariance update
```

Add numerical stability protections.

Use appropriate covariance handling.

Test:

- stationary case,
- constant velocity,
- acceleration,
- GNSS correction,
- GNSS outage,
- GNSS recovery.

Compare EKF output against controlled synthetic trajectories.

12. MOVE THE FULL NAVIGATION ENGINE TO EDGE

The current architecture must not rely on Python during live operation.

Implement the full navigation-critical computation on edge.

Preferred architecture:

```
ONNX TCN
+
Edge Navigation Engine
+
EKF
+
NHC
+
ZUPT
```

Use:

```
C++ → WebAssembly
```

or:

```
Rust → WebAssembly
```

if appropriate for the existing repository.

Do not duplicate navigation logic unnecessarily between Python and JavaScript.

Create one authoritative navigation implementation and use equivalent/reference implementations for testing where necessary.

13. FIX GNSS OUTAGE DETECTION

Do not use only:

```
if GPS == unavailable
```

Implement a GNSS quality/state machine:

```
GNSS-AIDED
  ↓
GNSS-DEGRADED
  ↓
GNSS-DENIED / DR
  ↓
GNSS-REACQUISITION
  ↓
GNSS-AIDED
```

Use available GNSS quality indicators.

Prevent rapid oscillation between states.

Use hysteresis and appropriate temporal thresholds.

Log every state transition.

14. FIX GNSS REACQUISITION

Never simply replace DR position with GNSS position.

Example:

```
DR = 1018 m
GNSS = 1000 m
```

must NOT cause:

1018 → 1000

as an instantaneous UI jump.

Instead:

DR estimate
↓
GNSS innovation
↓
EKF correction
↓
smooth continuous trajectory

Test large and small DR/GNSS discrepancies.

15. IMPLEMENT ROBUST NHC

For a ground vehicle, enforce appropriate motion constraints.

At minimum consider:

lateral velocity ≈ 0
vertical velocity ≈ 0

when the vehicle assumptions justify them.

Do not make constraints infinitely strong.

Use measurement covariance/confidence.

Relax constraints when appropriate for unusual motion.

Test:

- straight road,
 - turning,
 - acceleration,
 - braking,
 - rough road.
-

16. FIX ZUPT

Do not trigger ZUPT merely because acceleration is small.

Build a stationary detector using multiple signals, such as:

accelerometer variance
+
gyroscope magnitude/variance
+
estimated velocity
+
duration

Only apply ZUPT when stationary confidence is sufficiently high.

Test:

actual stop
constant-speed movement
slow movement
rough road

to prevent false ZUPT events.

17. IMPLEMENT OFFLINE MAP MATCHING

Runtime map matching must work without internet.

Use locally available road-network information.

Candidate matching can use:

position distance
heading compatibility
road direction
road geometry
previous road state
vehicle motion
turn constraints

Avoid simply snapping every position to the nearest road.

Implement trajectory-aware matching.

Compare:

DR without map matching
vs
DR with map matching

and quantify improvement.

18. HANDLE GNSS-DENIED OPERATION

When GNSS is available:

GNSS + IMU



AI velocity



EKF



NHC/ZUPT



Map Matching

When GNSS is denied:

IMU



AI velocity



EKF prediction



NHC/ZUPT



Dead Reckoning



Map Matching

When GNSS returns:

DR



GNSS validation



EKF correction



continuous navigation

The transition must be automatic.

19. BUILD A REAL EVALUATION FRAMEWORK

This is mandatory.

Create a reproducible benchmark that artificially removes GNSS from known trajectories.

Evaluate:

10 m

25 m

50 m

100 m

250 m

500 m

1 km

where data permits.

For every outage calculate:

absolute position error
maximum error
mean error
RMSE
final displacement error
velocity error
heading error
drift percentage

Calculate:

$$\text{drift_percentage} = \frac{\text{position_error}}{\text{distance_travelled}} \times 100$$

20. TARGET PERFORMANCE

The documented project targets are:

Overall drift:
< 10%

50 m GNSS-denied travel:
< 5 m drift

1 km GNSS-denied travel:
< 100 m drift at 60 km/h

Smartphone:
10 Hz

External FOG IMU:
~200 Hz or higher

These are targets, not assumed achievements.

Do not claim a target is achieved until benchmark results prove it.

21. PERFORM A COMPONENT ABLATION STUDY

This is extremely important.

Evaluate:

- A. Raw IMU DR
- B. AI only
- C. AI + EKF
- D. AI + EKF + NHC
- E. AI + EKF + NHC + ZUPT
- F. AI + EKF + NHC + ZUPT + Map Matching

Generate a table:

Component	Error

Raw IMU	X
AI	X
AI + EKF	X
AI + EKF + NHC	X
AI + EKF + NHC + ZUPT	X
Full system	X

This proves that each subsystem contributes measurable value.

22. TEST REAL-WORLD ROBUSTNESS

If real vehicle data exists in the repository, use it.

If not, prepare the codebase so real field data can be collected and inserted without modifying the navigation engine.

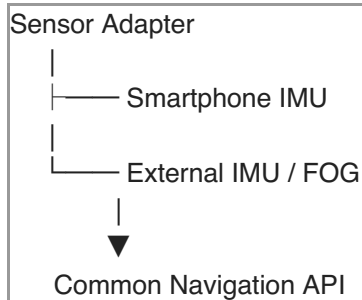
Test:

- different vehicles,
- different phones,
- different mounting orientations,
- dashboard mounting,
- pocket/hand movement where relevant,
- smooth roads,
- rough roads,
- acceleration,
- braking,
- turns,
- stops,
- tunnels,
- underpasses,
- urban environments.

Do not claim real-world performance without real-world data.

23. EXTERNAL IMU SUPPORT

Create a common sensor interface:



The navigation engine should not depend on a particular sensor source.

Support approximately:

Smartphone: 10 Hz+
External IMU: ~200 Hz+

Benchmark the actual achievable rate.

24. PWA / OFFLINE REQUIREMENTS

Verify that the PWA works when:

Wi-Fi = OFF
Cellular = OFF
Internet = OFF

After installation/cache preparation, verify:

application loads
model loads
ONNX inference works
sensor capture works
navigation works
map data works
UI works

Do not allow runtime dependencies on remote APIs.

Inspect the Service Worker carefully.

Make sure model and required assets are cached correctly.

25. EDGE PERFORMANCE BENCHMARK

Measure on an actual target phone:

TCN inference latency EKF latency NHC latency map matching latency total loop latency CPU usage RAM usage model size startup time battery consumption if measurable
--

For 10 Hz operation:

available cycle budget $\approx$ 100 ms

Keep sufficient margin.

Do not report theoretical performance.

Report measured performance.

26. CREATE AUTOMATED TESTS

Maintain and expand the existing test suite.

The project currently documents 51/51 passing tests.

Do not break them.

Add tests for:

coordinate transforms orientation sensor synchronization preprocessing TCN input/output shape ONNX parity EKF NHC ZUPT GNSS outage GNSS recovery map matching trajectory continuity edge inference offline execution
--

Run the complete test suite after every major change.

27. ONNX PARITY TEST

The PyTorch and ONNX versions must produce sufficiently equivalent outputs.

For identical input:

PyTorch output vs ONNX output

compare:

absolute difference relative difference
--

Define an acceptable numerical tolerance.

If parity fails, investigate before deployment.

28. UI REQUIREMENTS

The UI should clearly display:

Navigation Mode: GNSS / DEGRADED / DEAD RECKONING / REACQUISITION Speed Heading Latitude Longitude Position uncertainty GNSS status DR duration Drift estimate

Show:

Ground Truth vs Navigators trajectory

in simulator/evaluation mode.

Do not allow the UI to hide errors.

The purpose of the simulator is technical validation, not only demonstration.

29. LOGGING

Implement structured logs for:

timestamp
sensor rate
GNSS status
navigation mode
AI velocity
EKF state
uncertainty
NHC status
ZUPT status
map-match result
position
heading
speed
latency

Make logs exportable for offline analysis.

30. DOCUMENTATION

After implementation, update documentation to accurately reflect:

What is implemented
What is tested
What is experimentally verified
What remains future work

Do not state:

“professional-grade accuracy”

unless experimental evidence supports it.

Do not state:

“meets the target”

unless benchmark results demonstrate it.

Clearly separate:

Requirement
Implementation
Measurement
Result

31. DO NOT MAKE THESE MISTAKES

Do NOT:

- hard-code accuracy numbers,
- fabricate benchmark results,

- fabricate field-test results,
 - remove failing tests instead of fixing code,
 - silently catch errors,
 - hide numerical instability,
 - replace GNSS with DR using a hard position jump,
 - assume phone orientation is fixed,
 - assume low acceleration means stationary,
 - blindly snap positions to the nearest road,
 - depend on a backend during offline navigation,
 - optimize only model loss,
 - randomly split overlapping trajectory windows,
 - remove functionality simply to make tests pass,
 - rewrite working modules unnecessarily.
-

32. DEVELOPMENT STRATEGY

Work in this exact order:

Phase A — Audit

Inspect everything.

Produce a gap report.

Phase B — Baseline

Run the existing system.

Generate baseline metrics.

Phase C — Sensor pipeline

Fix:

timestamps calibration alignment orientation filtering
--

Phase D — ML

Improve and validate the TCN.

Phase E — Navigation

Fix:

EKF NHC ZUPT DR

Phase F — GNSS state management

Implement robust:

GNSS degraded denied reacquisition

Phase G — Map matching

Implement offline trajectory matching.

Phase H — Edge

Move navigation-critical processing to WebAssembly/edge.

Phase I — Benchmark

Run systematic GNSS outage experiments.

Phase J — Field validation

Validate on real vehicle data.

Phase K — Optimization

Optimize latency, CPU, RAM, battery, and model size.

Phase L — UI/demo

Polish only after technical correctness is established.

33. REQUIRED OUTPUT AFTER EACH MAJOR PHASE

For every phase, report:

1. Files changed
2. What was wrong
3. What was fixed
4. Why the fix is correct
5. Tests added
6. Tests passed
7. Benchmark results
8. Remaining limitations

Never simply say:

“Done.”

Provide evidence.

34. FINAL ACCEPTANCE CRITERIA

The project should be considered complete only when:

Navigation

- AI velocity estimation works.
- Phone-to-vehicle alignment works.
- IMU preprocessing is robust.
- EKF works correctly.
- NHC works correctly.
- ZUPT works correctly.
- GNSS outage detection works.
- GNSS recovery works without trajectory jumps.
- Dead reckoning works.
- Offline map matching works.

ML

- Model training is reproducible.
- Test split avoids trajectory leakage.
- TCN/ONNX outputs are validated.
- Edge inference is benchmarked.

Edge

- No backend is required for live navigation.
- PWA can operate offline after installation/cache preparation.
- ONNX runs locally.

- Navigation engine runs locally.
- 10 Hz smartphone operation is demonstrated.

Accuracy

Demonstrate actual measurements against:

< 5 m drift / 50 m outage

< 100 m drift / 1 km outage at 60 km/h

< 10% overall drift target

If a target is not achieved, DO NOT fake it.

Instead:

1. identify the failure,
2. determine the dominant error source,
3. implement a justified improvement,
4. rerun the benchmark,
5. document the result.

35. FINAL DELIVERABLE

At the end, produce:

A. Architecture diagram

Final production architecture.

B. Gap report

Before vs after.

C. Test report

All tests and results.

D. ML report

Training configuration and evaluation.

E. Navigation report

EKF/NHC/ZUPT/DR validation.

F. GNSS outage report

50 m, 100 m, 250 m, 500 m, 1 km experiments.

G. Edge benchmark

Latency, CPU, RAM, model size, update rate.

H. Field-test report

Real-world results, if available.

I. Known limitations

Be completely honest.

J. SIH-ready metrics

Provide a concise table:

Metric	Result	Target	Status

50 m DR error	___ m	<5 m	___
1 km DR error	___ m	<100 m	___
Overall drift	___ %	<10%	___
Update rate	___ Hz	10 Hz	___
Edge inference	___ ms	<100 ms	___
Model size	___ MB	—	—
CPU usage	___ %	—	—
Offline operation	YES/NO	YES	___
GNSS recovery	YES/NO	YES	___
Map matching	YES/NO	YES	___
External IMU	YES/NO	~200 Hz	___

FINAL INSTRUCTION

Start by auditing the existing repository.

Do NOT immediately rewrite everything.

Find the actual weaknesses first.

Then fix the highest-impact problems in priority order.

After every significant modification:

```
run tests
run static checks
run the relevant benchmark
inspect the output
```

Preserve working functionality.

Prefer scientifically measurable improvements over speculative complexity.

The ultimate objective is not merely to produce a sophisticated codebase.

The objective is to produce a **working, measurable, reproducible, offline Intelligent Dead Reckoning system capable of approaching or achieving the documented Navigators performance targets under GNSS-denied conditions.**